

Senior Software Engineer - Take Home Assessment

Time Estimate: 3-5 hours

Technologies: Python (any framework), React, any database

Overview

You'll be building a **Patient Care Dashboard** - a care management system that helps Care Admins manage patients, their care teams, and track health screening scores over time. This assessment evaluates your ability to design systems, make architectural decisions, and build production-ready features.

Problem Statement

Cerula Care's Care Admins need a dashboard to manage their workflow. They work with:

- **Patients** who have basic demographic information, are enrolled in care programs, and have different statuses (active, inactive, discharged)
- **Care Team Members** who can be assigned to patients. These include Health Coaches, Behavioral Health Care Managers (BHCM), and Psychiatrists
- **Health Screenings** that track behavioral health scores over time

Understanding Health Screenings

Patients complete a health screening each month. Health screenings produce a score from 0-10, with higher scores indicating more severe symptoms. Screenings are completed monthly (one screening per patient per month) and are used to track behavioral health status over time and monitor patient progress.

Key Points:

- Health screenings are pre-stored in your database - you don't need to build functionality to create new screenings
- Include health screening data for the **last 6 months** in your seed data (so patients have historical data to visualize)
- Care Admins need to see how Health Screening Scores change over time to monitor patient progress. Lower scores generally indicate improvement.

User Needs:

1. Care Admins need to view and manage a list of patients
2. They need to assign and unassign care team members to/from patients
3. They need to view patient details and see how health screening scores change over time
4. They need to be able to search and filter patients efficiently

Your Task: Design and build a working full-stack application that addresses these needs. You'll need to make decisions about:

- Database schema and relationships
 - API structure and endpoints
 - Frontend architecture and user experience
 - Data validation and error handling
 - How to present health screening trends visually
-

Requirements

What Must Work

The application must be **fully functional and testable**. A reviewer should be able to:

1. **View Patients**
 - See a list of patients with relevant information
 - Search/filter patients (you decide what fields are searchable and how)
 - Navigate through multiple patients if there are many (you decide pagination approach)
2. **Manage Patient Details**
 - View detailed information about a specific patient
 - Edit patient information
 - Create new patients
3. **Manage Care Team Assignments**
 - View which care team members are assigned to a patient
 - Assign care team members to patients
 - Unassign care team members
4. **Track Health Screenings**
 - View health screening history for a patient
 - Visualize health screening trends over time (you choose the visualization approach)

Important: While you have design freedom, the application must be **runnable and testable**.
Include:

- Clear setup instructions
- Database migrations/schema setup
- Seed data with patients, care team members, assignments, and **monthly health screenings for the last 6 months** for each patient
- Both backend and frontend must start and work together

Technical Constraints

- **Backend:** Python with any framework, any database
- **Frontend:** React (TypeScript preferred but not required)
- **Must be runnable:** Include setup instructions and seed data so reviewers can test it
- **Production-minded:** Handle errors gracefully, validate inputs, consider edge cases

Design Freedom

You have full autonomy to make design decisions about database schema, API structure, frontend architecture, and user experience.

Deliverables

1. Working Application

- Backend API that can be started and tested
- Frontend that can be started and used
- Database with schema and seed data
- Everything must be runnable by a reviewer

2. Documentation

- README.md with clear setup instructions
- Explanation of your design decisions and tradeoffs
- API documentation (many frameworks auto-generate this, but you may want to add notes)
- Any assumptions you made or limitations

3. Code Quality

- Well-organized, readable code
 - Clear project structure
 - Proper dependency management (requirements.txt, package.json, etc.)
-

Submission

Please provide:

1. A link to your Git repository (GitHub, GitLab, etc.) **OR** a zipfile of your project
2. Instructions on how to run the project locally
3. Any additional notes or explanations you'd like to share

If submitting a zipfile:

- Include all project files (backend, frontend, database migrations, seed data, etc.)
- Provide clear setup instructions in a README.md file
- Ensure the zipfile can be extracted and the project can be set up and run following the instructions
- Include all necessary configuration files and dependencies